

PIC-tutor

从动理学模型到 WarpX 源码的 Particle-In-Cell 讲解

原生 LaTeX 版本 (Phase 1 样章)

基线书稿: v0.110 (Git 标签 v1.0)

PIC-tutor 项目

2026

目录

写作说明	1
0.1 这本书为谁而写	1
0.2 如何使用本书	1
0.2.1 四个跨章节术语	2
0.3 按目标选择阅读路线	2
0.3.1 路线 A: 先能运行并解释一个最小 PIC case	2
0.3.2 路线 B: 先读懂一条从粒子到场的源码链	3
0.3.3 路线 C: 先学会设计验证, 而不是只收集输出	3
第一章 动理学模型与 PIC 的基本思想	4
1.1 Vlasov 方程首先是相空间守恒律	4
1.2 碰撞项不是 Vlasov 的一部分, 但必须保留边界	5
1.3 Vlasov-Maxwell: 源项、约束与闭合	6
1.4 Vlasov-Maxwell 的能量与动量守恒边界	7
1.5 Vlasov-Poisson / electrostatic 极限不是另一套世界	7
1.5.1 模型选择与验证卡: Poisson 可解不等于完整电磁问题已被解决	8
1.6 宏粒子不是假粒子, 而是 coarse-grained 分布函数载体	9
1.7 权重可以不同, 但不是没有代价	9
1.8 shape factor 不是插值细节, 而是粒子-网格耦合规则	10
1.9 PIC 的噪声不是 bug, 而是模型代价	11
1.9.1 统计噪声与能量账本验证卡: 能量漂移小不等于热平衡或低噪声	12
1.10 Debye 长度、粒子数与统计时间尺度	13
1.10.1 尺度计算卡: 能量回归通过不等于 Debye 屏蔽已分辨	13
1.11 从连续模型到 PIC 离散变量	14
1.12 这一章对后面源码章节的真正约束	15
1.12.1 跨章交接卡: 从连续对象进入离散循环	16
1.13 证据范围与继续阅读	17
1.14 练习与源码定位	17
第二章 PIC 总循环: 从 Vlasov-Maxwell 到离散时间推进	18
2.0.1 从第 1 章进入本章: 先写状态, 再读调用顺序	18
2.1 连续模型: Vlasov-Maxwell 系统	18

2.2	宏粒子表示与形函数	19
2.3	leapfrog 时间层	20
2.3.1	ω_p 不是背景常数, 而是时间离散必须尊重的最快等离子体尺度	21
2.3.2	λ_D 不只是一条长度定义, 它直接约束 Δx	21
2.4	FDTD 场更新的数学骨架	22
2.4.1	CFL 不是经验参数, 而是离散 Maxwell 更新的因果上界	23
2.4.2	数值色散从这里开始: 连续光锥被离散 stencil 改写	24
2.4.3	Yee / Nodal / CKC 的差别本质上是离散色散关系不同	25
2.5	PSATD 与 FDTD 的主循环差异	26
2.6	一个完整 PIC step 的离散组织	27
2.6.1	AMR subcycling: 两个时间步不是同一个时间步的重复调用	28
2.6.2	JRhom 与 implicit: 同一个外层 step 内部也可能有不同的时间合同	29
2.7	本章后的源码阅读入口	30
2.8	参数示例与最小运行案例	31
2.8.1	第一次运行的读者路线: 构建、CTest 与手动分析各自回答什么	32
2.8.2	受控修改路线: 一个命令行覆盖会改变哪一份证据	33
2.9	基础文献与证据范围	34
2.10	进一步阅读与练习	35
2.11	本章结论	35

写作说明

0.1 这本书为谁而写

如果你会运行一个 PIC case，却还不能回答“粒子在什么时间层被推进”“电流为什么要沿轨迹沉积”“一个小的 $\text{divE}-\rho$ 残差究竟说明什么”，这本书就是从这里开始的。它不要求读者先熟悉 WarpX 的类层次，也不把源码中的函数名当作知识本身；每个实现细节都先放回物理量、离散方程和可观察结果中解释。

读完主线后，读者应当能够：

- 从连续方程推导出 PIC 中需要保存和交换的离散量；
- 沿着一个输入参数追踪到初始化、时间推进、沉积、场更新和诊断输出；
- 为一个新 case 选择时间步、网格、shape、solver 和诊断，而不是只复制输入文件；
- 看见 regression 通过时，准确说出它证明了什么、还没有证明什么。

0.2 如何使用本书

正文按“模型 -> 离散算法 -> 源码调用链 -> 案例诊断”的顺序展开。章节末的练习要求读者自己定位源码、检查时间层或复现实验；它们不是附加的项目验收清单，而是把阅读变成判断能力的训练。

版本号、运行合同和缺口登记记录的是书稿如何被维护，不是读者必须按时间顺序阅读的内容。正文只在需要解释证据范围时使用它们，不把它们作为学习前提。

本书不是 WarpX 官方文档的翻译，也不是只讲公式的 PIC 理论笔记。它的主线是：先从 Vlasov-Maxwell / Vlasov-Poisson 这类连续模型出发，说明为什么需要宏粒子；再把宏粒子、网格、形函数、沉积和场求解拼成 PIC 算法；最后回到 WarpX 的 Source/、Docs/、Examples/ 和 regression 入口，解释一个现代高性能 PIC 程序如何把这些步骤组织成可运行、可扩展、可验证的模拟软件。

源码定位应优先依赖文件职责、函数名和调用关系，而不是固定 commit 或行号。例如，主循环从 `Evolve()` 与 `OneStep()` 开始，初始化从 `InitData()` 开始，粒子/源项主链从 `PushParticlesandDeposit()` 与 `SyncCurrentAndRho()` 开始。版本信息只用于复现实验和维护记录，不应改变读者对算法关系的理解。

本书的每个技术判断都应尽量落到六类证据：物理方程、离散公式、WarpX 源码路径、输入参数、示例或测试、文献。DeepWiki、Zread 等 AI 解读页面可以用来快速找到模块名，但不能作为最终依据。

遇到一个新的输入或源码分支时，可按四步阅读：先写出要描述连续物理量；再标出它在离散网格和时间层上的表示；随后定位 producer 和 consumer；最后选择一个能区分正确

与错误的 observable，并写下该观察量不能支持的外推。这样，代码阅读始终服务于物理判断，而不是变成函数名清单。

0.2.1 四个跨章节术语

后续章节反复使用下列词，但它们不是函数名、文件名或测试标签的同义词：

术语	本书中的含义	阅读时必须继续追问	不能据此直接推出
source	被某个场或方程更新实际消费的离散源项或状态，例如同步后的 ρ 、 $\mathbf{J}$ 或谱空间历史量	它在哪个时间层、几 何位置、AMR level 和同步阶段成立	任意名为 <code>rho</code> 、 <code>current_*</code> 的数组都已可供 solver 消费
producer	在确定的生命周期阶段创建、推进、沉积、同步或归约某个状态的代码路径	它生成什么量、何时 写入、是否还需要过 滤、通信或边界处理	找到一次写入或对象构造就证明物理状态已经完整
consumer	读取该状态以更新场、写诊断、作比较或产生后续状态的代码路径	它读取的是 producer 的哪一时间层和哪一 种表示	输出文件成功写出就证明 producer 的物理语义正确
observable	为回答一个物理问题而定义、可与解析解、守恒关系、reference 或实验量比较的量	它的比较对象、容 差、几何和不可外推 范围是什么	任意已输出字段、 <code>checksum</code> 或图像都是物理验证

“证据层级”则描述上述链条有多强：源码可以定位职责，指定案例可以检验给定条件下的 observable，全文文献可以支撑公式或机制；它们必须互相独立，不能把其中任一层放大成其余各层。第 3 章把这四个术语接到控制流和状态交接，第 5–7 章说明 source 怎样经过推进、同步和边界，第 8 章说明 consumer 如何形成诊断，第 9 章再判断外部 reference 的证据强度。

本书默认使用以下记号：粒子位置为 $\mathbf{x}_p$ ，粒子动量为 $\mathbf{u}_p = \gamma \mathbf{v}_p$ ，电磁场为 $\mathbf{E}, \mathbf{B}$ ，电荷和电流密度为 $\rho, \mathbf{J}$ ，粒子权重为 w_p ，形函数为 S 。网格量的上标表示时间层，例如 $\mathbf{B}^{n+1/2}$ ；粒子量一般按 leapfrog 交错在位置和动量时间层上。

0.3 按目标选择阅读路线

不必把全书当作一条只能从头到尾走完的线。先选择你要获得的能力，再沿相应路线完成每个终点练习；三条路线共享术语和证据边界，但不能用其中一条的通过替代另一条的结论。

0.3.1 路线 A：先能运行并解释一个最小 PIC case

适合已经会启动程序、但还不能说明输出为何可信的读者。先读第 1.10–1.14，写出 Debye 长度、时间尺度和一个要保留的 observable；再读第 2.6–2.10，把它放进一个完整 PIC step 和最小输入；接着用第 3.12–3.13 追踪输入、`warpx_used_inputs`、输出与 analysis 的交接；最后完成第 8.15 的 Langmuir 或 uniform-plasma 练习。终点不是“作业跑完”，而是能写出该 case 的 producer、consumer、比较量、容差以及它不能覆盖的 geometry、算法和物理范围。

0.3.2 路线 B：先读懂一条从粒子到场的源码链

适合需要修改或审查实现、但不希望先被所有模块细节淹没的读者。以第 2.7 的导航为索引，读第 3.4–3.10 确定初始化、外层时间步和 source 同步的职责；然后依次完成第 4.16、5.16、6.12 的练习，分别追踪粒子状态、 ρ/J 和场更新的时间层。每次定位函数时先写“它消费什么、产生什么、下一位 consumer 是谁”，再阅读分派或 kernel；不能从找到一次数组写入就推断求解器已经消费了物理上完整的 source。

0.3.3 路线 C：先学会设计验证，而不是只收集输出

适合准备建立新 case、改变数值算法或解释异常结果的读者。先读第 1.5.1 与第 1.9.1，区分模型选择、能量账本和统计量；再读第 5.14.2.3 与第 5.14.7，把 source residual、解析场、checksum 和收敛趋势拆成不同 consumer；随后以第 8.14–8.16 设计一张诊断记录卡；最后用第 9.8–9.10 复核文献、源码和运行证据能否支持措辞。终点是一份可审查的验证合同，而不是一张看起来合理的场图或一次 regression PASS。

第一次阅读仍可先走路线 A，再按需要进入 B 或 C。无论选择哪条路线，遇到一个新的输入或源码分支，都回到本节前的四步：连续物理量、离散表示、producer/consumer 和带范围说明的 observable。

第一章 动理学模型与 PIC 的基本思想

PIC 代码不是先有“粒子数组”和“场数组”，再去给它们找物理意义。它的上游模型是动理学方程：对每个物种，真实对象首先是相空间分布函数

$$f_s(\mathbf{x}, \mathbf{p}, t),$$

而不是单个粒子轨道。本章的任务是把这条连续模型主线压实到后面源码会反复用到的几个边界：

1. Vlasov 方程本质上是相空间守恒定律，而不只是“一个偏微分方程”。
2. Vlasov-Maxwell 与 Vlasov-Poisson 不是两套无关模型，而是同一条自洽场闭合在不同物理极限下的分支。
3. 宏粒子、权重和 shape factor 不是数值技巧附会到物理上，而是 coarse-grained kinetic model 的一部分。
4. PIC 的主要误差不是单一来源，而是采样噪声、有限粒子权重、有限网格和离散时间层共同作用的结果。

本章的读者路线

不要把本章当作术语表。先在 1.1–1.4 固定连续对象：什么量在相空间中守恒、哪些矩成为场的 source、能量和动量何时可以交换；再在 1.5 判断问题是否真的允许采用静电近似；随后在 1.6–1.10 判断宏粒子、权重、shape、网格和时间平均会怎样改变可见的统计量；最后用 1.11–1.14 把这些连续对象交给第 2 章的离散时间步。每经过一段，都应能回答三件事：程序实际表示什么、与哪一个连续量比较、该比较还不能保证什么。

本章的阅读支点是：

- Birdsall 1985
- Dawson 1983
- 第 2、3、5、6 章对 WarpX 主循环、沉积和场求解器的实现说明

Hockney–Eastwood 与 Yee 1966 在本书中只作为补充书目信息，而不作为可逐页核对的全文依据；本章不把无法核实的历史细节当作结论。读者应先把这里的连续模型、离散变量和误差边界读清，再进入源码章节。

1.1 Vlasov 方程首先是相空间守恒律

对物种 s ，若忽略碰撞、衰变、电离和其他 source/sink，分布函数满足无碰撞相对论 Vlasov 方程

$$\frac{\partial f_s}{\partial t} + \dot{\mathbf{x}} \cdot \nabla_{\mathbf{x}} f_s + \dot{\mathbf{p}} \cdot \nabla_{\mathbf{p}} f_s = 0.$$

这里采用的归一化是

$$dN_s = f_s(\mathbf{x}, \mathbf{p}, t) d\mathbf{x} d\mathbf{p}, \quad n_s(\mathbf{x}, t) = \int f_s(\mathbf{x}, \mathbf{p}, t) d\mathbf{p}.$$

因此, f_s 不是“某个粒子的概率标签”: 它是对一个相空间体元内物种数目的密度。后面用粒子样本近似它时, 必须同时说明粒子位置、动量、权重和 shape; 只写出一条轨道不能定义 f_s 。

对带电粒子,

$$\dot{\mathbf{x}} = \mathbf{v}, \quad \dot{\mathbf{p}} = q_s (\mathbf{E} + \mathbf{v} \times \mathbf{B}),$$

因此可写成更常见的形式

$$\frac{\partial f_s}{\partial t} + \mathbf{v} \cdot \nabla_{\mathbf{x}} f_s + q_s (\mathbf{E} + \mathbf{v} \times \mathbf{B}) \cdot \nabla_{\mathbf{p}} f_s = 0.$$

这里真正需要记住的不是式子本身, 而是它的守恒含义。把相空间速度记成

$$\mathbf{Z} = (\mathbf{x}, \mathbf{p}), \quad \dot{\mathbf{Z}} = (\dot{\mathbf{x}}, \dot{\mathbf{p}}),$$

则 Vlasov 方程也可写成

$$\frac{\partial f_s}{\partial t} + \nabla_{\mathbf{Z}} \cdot (f_s \dot{\mathbf{Z}}) = 0.$$

若相空间流满足

$$\nabla_{\mathbf{Z}} \cdot \dot{\mathbf{Z}} = 0,$$

就得到 Liouville 图像: 沿特征线传播时, 分布函数值保持不变, 相空间体积也不被压缩或膨胀。对后面的 PIC 来说, 这一点比公式本身更基础, 因为它解释了为什么:

- 粒子推进器不能随意破坏轨道拓扑;
- 离散时间推进若不尊重相空间结构, 就会把数值伪耗散或伪加热写进分布函数;
- Boris / Vay / Higuera-Cary 这类 pusher 的比较, 不只是在比轨道误差, 而是在比它们怎样离散化相空间流。

1.2 碰撞项不是 Vlasov 的一部分, 但必须保留边界

一旦考虑碰撞、衰变、外部源项或粒子数变化, 更一般的 kinetic equation 应写成

$$\frac{\partial f_s}{\partial t} + \mathbf{v} \cdot \nabla_{\mathbf{x}} f_s + q_s (\mathbf{E} + \mathbf{v} \times \mathbf{B}) \cdot \nabla_{\mathbf{p}} f_s = C_s[f] + S_s - L_s.$$

这里:

- $C_s[f]$ 表示碰撞算子，可以是 Boltzmann、Landau、Fokker-Planck 或 Monte-Carlo 近似；
- S_s 表示外加源项，如电离生成、注入、衰变产物；
- L_s 表示损失项，如吸收、衰变消失、边界流出。

这条边界在 WarpX 里很重要，因为：

- 主 PIC loop 默认走的是无碰撞 Vlasov-Maxwell 主线；
- CollisionHandler、field ionization、QED、ContinuousFluxInjection、粒子边界吸收/反射，都是往这条无碰撞主线上附加 C/S/L；
- 它们不该被写成“另一个独立程序”，而应被理解成对同一 kinetic balance 的修改。

因此本书后面凡是讲 collisions、ionization、QED、scraping，都应问两个问题：

1. 它改的是分布函数右端的哪一项？
2. 它是在一个时间步的哪个时间层插入进去？

否则很容易把“物理过程存在”和“离散时间组织正确”混成一件事。

1.3 Vlasov-Maxwell：源项、约束与闭合

相对论动量与速度满足

$$\mathbf{p} = \gamma m_s \mathbf{v}, \quad \gamma = \sqrt{1 + \frac{|\mathbf{p}|^2}{m_s^2 c^2}}, \quad \mathbf{v} = \frac{\mathbf{p}}{\gamma m_s}.$$

分布函数的低阶矩给出 Maxwell 方程右端的源项：

$$\begin{aligned} \rho(\mathbf{x}, t) &= \sum_s q_s \int f_s(\mathbf{x}, \mathbf{p}, t) d\mathbf{p}, \\ \mathbf{J}(\mathbf{x}, t) &= \sum_s q_s \int \mathbf{v}(\mathbf{p}) f_s(\mathbf{x}, \mathbf{p}, t) d\mathbf{p}. \end{aligned}$$

然后场满足

$$\begin{aligned} \nabla \cdot \mathbf{E} &= \frac{\rho}{\epsilon_0}, \quad \nabla \cdot \mathbf{B} = 0, \\ \nabla \times \mathbf{E} &= -\frac{\partial \mathbf{B}}{\partial t}, \quad \nabla \times \mathbf{B} = \mu_0 \mathbf{J} + \frac{1}{c^2} \frac{\partial \mathbf{E}}{\partial t}. \end{aligned}$$

这四式里最容易被误写的是：Gauss 定律和 $\text{div } \mathbf{B} = 0$ 不是“额外条件”，而是系统闭合的一部分。只要连续性方程

$$\frac{\partial \rho}{\partial t} + \nabla \cdot \mathbf{J} = 0$$

成立，并且初值满足约束方程，Maxwell 演化就会传播这些约束。反过来，如果离散沉积和离散场推进不一致，那么程序里最先坏掉的往往不是 curl 更新，而是：

- $\text{divE} = \rho / \epsilon_0$
- divB
- 边界附近的伪电荷
- 以及随之而来的非物理电场和数值加热

这正是后面第 5 章和第 6 章为什么要反复围着 source synchronization、current correction、Gauss-law regression 打转。

1.4 Vlasov-Maxwell 的能量与动量守恒边界

在闭域、无外源、忽略边界通量时，连续系统满足总能量守恒：

$$\frac{d}{dt} \left[\sum_s \int \gamma m_s c^2 f_s d\mathbf{x} d\mathbf{p} + \int \left(\frac{\epsilon_0}{2} |\mathbf{E}|^2 + \frac{1}{2\mu_0} |\mathbf{B}|^2 \right) d\mathbf{x} \right] = 0.$$

它说明两件事：

1. 粒子动能和场能不是分开各自守恒，而是可以相互交换。
2. 程序里若只监控粒子能量或只监控场能量，都不足以判断离散系统是否健康。

同理，总动量守恒应理解为“粒子动量 + 电磁场动量 + 边界 Maxwell stress”一起守恒，而不是单看粒子束团动量曲线。对后面的 implicit、hybrid、electrostatic sphere、planar pinch 和 FEL 例子，这个边界都非常关键：自动化案例即使检查完整能量账本，也不能由某个单独变量“看起来没漂”来替代。

1.5 Vlasov-Poisson / electrostatic 极限不是另一套世界

当系统关注的是电荷分离和纵向静电响应，而电磁波传播、辐射和横向磁反馈不是主导效应时，可以把自洽场闭合约化到 Vlasov-Poisson：

$$\frac{\partial f_s}{\partial t} + \mathbf{v} \cdot \nabla_{\mathbf{x}} f_s + q_s \mathbf{E} \cdot \nabla_{\mathbf{p}} f_s = 0,$$

$$\mathbf{E} = -\nabla \phi, \quad -\nabla^2 \phi = \frac{\rho}{\epsilon_0}.$$

它不是凭空把 Maxwell 方程换掉，而是对应这样一组物理假设：

- transverse electromagnetic radiation 不是主要自由度；
- 场主要由电荷分离决定；
- 电磁传播时间尺度不是当前主导尺度；
- 所关心的现象更接近 Langmuir、space-charge、electrostatic expansion、Poisson boundary-value problem。

因此：

- electrostatic PIC 仍然是 kinetic PIC；
- 只是“粒子如何给场提供源项、场如何回馈粒子”这一闭合从 Maxwell 变成了 Poisson。

这是一项建模假设，不是看到某一帧电场主要沿一个方向后再补上的解释。WarpX 的 `warpx.do_electrostatic` 参数说明也明确指出：该模式不在每一步用完整 Maxwell 方程更新场，传播的电磁辐射和激光等效效应不被捕获。因此，若研究目标包含传播、辐射、激光注入或横向电磁反馈，就不能用“当前图像看起来近静电”替代模型适用性的论证。

这也是为什么：

- 第 6 章不能把 electrostatic solver 当作“Maxwell solver 的低配版”；
- electrostatic sphere、Pierce diode、effective potential 这些例子要和 Poisson 边界条件、势能账本一起讲；
- `WarpX::OneStep()` 里 electrostatic / hybrid 路线的场解位置会和标准 electromagnetic loop 不同。

1.5.1 模型选择与验证卡：Poisson 可解不等于完整电磁问题已被解决

选择 electrostatic 路线时，先把“模型是否适合”与“这一实现是否通过指定验证”分开。下面的四层顺序可防止把一个收敛的 Poisson solve、一次通过的回归或一幅近纵向电场图误读成完整的 Vlasov–Maxwell 验证。

第一层：先判断问题是否仍需传播电磁自由度。若研究对象包含激光注入、辐射传播、波的有限传播时间、横向电磁反馈或与 PML 吸收有关的反射，`warpx.do_electrostatic` 不是近似程度较高的快捷路径，而是模型对象已经变了；应回到完整 electromagnetic 路线。反之，若目标是给定边界势下的空间电荷、静电平衡或 Poisson 边值问题，才继续问 Poisson 解和边界条件是否与目标相符。参数文档的“没有 timestep 限制”只是在说该模式不由 electromagnetic CFL 限制；它既不免除粒子输运、等离子体频率、统计采样或 Poisson 误差的判断，也不授权把过大的步长写成物理上准确。

第二层：明确实际解的是什么。labframe 以所有物种合成的 ρ 解

$$\nabla^2 \phi = -\rho/\epsilon_0, \quad \mathbf{E} = -\nabla \phi.$$

labframe–electromagnetostatic 还以合成的 $\mathbf{J}$ 解矢势并构造静磁场；labframe–effective–potential 则改写 Poisson 算子以改变大 $\omega_{pe} \Delta t$ 时的数值行为。它们不是同一个开关的三个拼写。读者若只需 electrostatic 的 $\phi, \mathbf{E}$ ，不能因为输出里出现了 B 或因为某个模式能稳定运行，就跳过所选模式的方程和适用假设。

第三层：用一个有解析 reference 的 producer 检查指定对象。官方 `test_3d_electrostatic_sphere_eb` 是启用 EB 时的 3D、2-rank 案例：`max_step = 1`、`warpx.do_electrostatic = labframe`、外边界为零电势的 PEC，半径 $R = 0.1\text{ m}$ 的 embedded sphere 固定为 $\phi_0 = 1\text{ V}$ 。它写出 `Ex/Ey/Ez/rho/phi/eb_covered` 的 Full diagnostics，并另写 `ChargeOnEB` 的全球与八分之一球积分。这里的 producer 很窄：它构造的是给定导体势、特定几何、单步、无传播辐射主张的 Poisson/EB 问题，而不是一般带粒子动力学的 plasma benchmark。

第四层：让 consumer 与所问问题一一对应。该案例的 `analysis.py` 比较全嵌入边界电荷

$$Q_{\text{th}} = 4\pi\epsilon_0\phi_0 R$$

与 `eb_charge.txt` 中的数值积分，并要求相对误差小于 **0.06**；它还把八分之一权重积分与 $Q_{\text{th}}/8$ 比较，并检查 `eb_covered` 在 $[0, 1]$ 内、球内带与球外带分别为 1 与 0。这三类断言分别约束全局边界电荷、加权表面积分和 EB 几何 mask。CTest 随后的 checksum 只回归指定输出基线，不能替代前三个比较。

从执行次序看，初始化阶段和每个 electrostatic 时间步都会经 `ComputeSpaceChargeField()` 组织 Poisson 场；进入该分支后，演化代码会重置 electrostatic 分量，再在粒子下一次 push 前准备可 gather 的场。与此同时，`OneStep_sub1()` 明确拒绝 electrostatic solver 与 AMR subcycling 的组合。因此下面的句子都不成立：“Poisson 残差小，所以激光传播也正确”、“sphere 的电荷通过，所以任意 EB 几何的场都正确”、“没有 electromagnetic CFL，所以时间步不再需要物理判断”，以及“这张单步、固定势测试证明了 electrostatic + subcycling 的路径”。

这张卡的实际交付是一张四行表：模型中保留/删去的自由度、输入 producer、各 consumer 的 reference 与阈值、以及本次结论不能外推的范围。这样才把第 1 章的连续假设接到后续第 3A 章初始化、第 6 章场求解和第 7 章 EB/PML 的不同问题上。

1.6 宏粒子不是假粒子，而是 coarse-grained 分布函数载体

PIC 的核心近似不是把等离子体变成少数真实粒子，而是用有限数量的宏粒子采样分布函数。形式上可写成

$$f_s(\mathbf{x}, \mathbf{p}, t) \approx \sum_{p \in s} w_p S_x(\mathbf{x} - \mathbf{x}_p(t)) S_p(\mathbf{p} - \mathbf{p}_p(t)).$$

这里：

- w_p 是宏粒子权重；
- S_x 是空间形函数；
- S_p 常在实际 PIC 中退化成粒子自身在动量空间的离散采样。

从 Dawson 1983 的角度，更准确的说法是：宏粒子不是“把许多真实粒子团成一个球”的形象化故事，而是 coarse-grained kinetic model 的载体。它的目标是：

1. 用有限自由度代表连续分布；
2. 保住真正重要的低阶矩和 collective behavior；
3. 接受某些细粒度 phase-space 结构会被采样误差和 coarse graining 吞掉。

1.7 权重可以不同，但不是没有代价

宏粒子并不必然等权。Dawson 1983 讨论了

$$q_i = -\alpha_i e, \quad m_i = \alpha_i m, \quad \frac{q_i}{m_i} = -\frac{e}{m}$$

这一类不同电荷和质量、但相同荷质比的电子群，并说明由它们组成的加权分布函数仍满足通常的 Vlasov 方程。

这条结论的意义是：

- weighted macroparticles 从一开始就是合法的 kinetic coarse graining；
- 它允许把 phase-space 分辨率集中到真正需要的区域；
- 但它也会引入新的统计与 collisional side effects。

所以“加权宏粒子”更准确的理解不是“自适应采样免费升级”，而是：

- 你获得了 phase-space resolution redistribution；
- 但要付出更复杂的噪声、散射和统计解释代价。

这条边界对后面理解 WarpX 中：

- species 权重
- Gaussian beam / flux injection
- collision/QED product creation
- reduced-dimension weighting compensation

都很重要。

1.8 shape factor 不是插值细节，而是粒子-网格耦合规则

若把粒子源项沉积到网格单元或网格点 i ，最基本的电荷密度形式是

$$\rho_i^n = \frac{1}{\Delta V_i} \sum_p q_p w_p S_i(\mathbf{x}_p^n).$$

场 gather 则用同一类 shape family 从网格插值回粒子位置：

$$\mathbf{E}_p^n = \sum_i S_i(\mathbf{x}_p^n) \mathbf{E}_i^n, \quad \mathbf{B}_p^n = \sum_i S_i(\mathbf{x}_p^n) \mathbf{B}_i^n.$$

但 shape factor 的意义远不止“双向插值”：

1. 它定义了宏粒子在空间上的 coarse-grained 电荷云。
2. 它决定了粒子-网格耦合的 stencil 宽度。
3. 它会系统改写短波 aliasing、self-force 和统计噪声。
4. 它直接影响 guard-cell 需求、通信宽度和算子局域性。

Birdsall 1985 与 Dawson 1983 的共同结论都指向这一点: finite-size particles 不是为了把图画得更平滑, 而是为了软化 point-charge 的短程奇异作用、压低非物理 collisionality, 并把系统真正保留成“长程 collective physics + 可控短程误差”。

这也是为什么后面的第 5 章必须把:

- shape factor
- charge/current deposition
- sampled density
- finite-grid effects
- aliasing

放在同一章里讲, 而不是把 shape factor 单独缩成一个插值小节。

1.9 PIC 的噪声不是 bug, 而是模型代价

把连续分布函数换成有限宏粒子之后, 最基本的代价就是采样噪声。它不是代码写坏了才出现, 而是:

- 有限粒子数
- 有限权重
- 有限网格
- 有限时间平均

共同带来的统计涨落。

从 Birdsall 1985 的 thermal-plasma 讨论看, 这种噪声不能只被理解成“粒子数不够大”。更准确的图像是:

1. sampled density 会生成 alias branches;
2. shape factor 会修改 fluctuation spectrum;
3. finite Δx 和 finite Δt 会把 continuum 改写成带离散谱结构和 effective transport 的系统;
4. 若离散耦合处理不好, 噪声会演化成 numerical heating、drag、diffusion, 甚至弱不稳定增长率误判。

因此, 本书后面凡是说“噪声更小”“结果更平滑”, 都不应只停在图像层, 而应继续问:

- 是 modal fluctuation level 变了?
- 是 alias branch 被压了?
- 还是只把可见图像平滑了, 但守恒与统计量并没有更好?

1.9.1 统计噪声与能量账本验证卡：能量漂移小不等于热平衡或低噪声

当修改了宏粒子数、shape、gather、滤波或时间步时，最容易发生的误读是：看到总能量曲线“很平”，便宣称热等离子体保持正确、噪声已经降低。二者回答的不是同一个问题。WarpX 的 `energy_conserving_thermal_plasma` 回归给出了一条很有用、但范围明确的能量账本路线。

第一层：先固定它实际产生了什么。官方 CTest 注册了 1D 和 2D 两个 producer，均使用两个 MPI rank；两份输入都是周期边界、`max_step = 500`、`warpx.do_electrostatic = labframe`、二阶 `algo.particle_shape = 2`、`algo.field_gathering = energy-conserving`，并关闭滤波。电子和质子以相同的常数密度、Gaussian 动量分布和 100 eV 温度初始化。1D 每个 cell 每个方向放 4 个宏粒子，2D 每个方向放 2 个；两者都每 100 步写一次 ParticleEnergy 与 FieldEnergy。这是一份特定离散设置下的周期、双物种、静电热等离子体账本，不是任意 thermal plasma 的定义。

第二层：再核对 **consumer** 到底比较了什么。ParticleEnergy 的第一个能量列是所有物种的加权相对论动能，

$$E_p = \sum_i w_i \left(\sqrt{|\mathbf{p}_i|^2 c^2 + m_i^2 c^4} - m_i c^2 \right),$$

而 FieldEnergy 的第一个能量列是各场分量构成的场能

$$E_f = \frac{1}{2} \sum_{\text{cells}} \left(\epsilon_0 |\mathbf{E}|^2 + \frac{|\mathbf{B}|^2}{\mu_0} \right) \Delta V.$$

该 `analysis.py` 读取两张 reduced-diagnostic 表的第 3 列，形成 $E = E_f + E_p$ ，并在所有已记录的時刻断言

$$|E(t) - E(0)| / E(0) < 0.003.$$

这既不是“能量只能增加 0.3%”，也不是“严格守恒”：代码取绝对值，所以正、负漂移都受约束；而阈值只约束 producer 所写出的采样时刻。`energy-conserving gather` 的名字也不能代替这条检查。官方算法说明将它定义为从 grid point 直接 gather，并说明相应的能量守恒性质只在 $\Delta t \rightarrow 0$ 的极限成立，有限时间步通常只是具有更好的相应守恒表现。

第三层：明确这张账本没有测量什么。它不计算温度矩、速度分布是否仍为 Maxwellian、密度或电场的 k -spectrum、alias branch、两点相关或 transport coefficient；也没有让 momentum-conserving、不同 particle shape 或不同 particle-per-cell 与当前设置进行对照。因此通过该 consumer 不能推出“噪声更低”“热平衡正确”“每个物种能量正确”“数值严格守恒”，更不能外推到非周期边界、AMR 接口、碰撞、激光传播或其他几何。规则网格上 gather/deposition 的配对性质也不能自动越过网格不规则性；官方 AMR 理论明确指出，规则性或对称性被破坏时，接口附近会出现净的 spurious self-force。

第四层：修改后重新建立两本账。若只改变输出格式或保存位置，先保证 E_f 与 E_p 仍在同一物理时刻、同一归一化下被读取，再继续使用能量 consumer。若改动 gather、shape、滤波、 Δt 、宏粒子数、温度/密度、边界、维度或求解模型，则原来的 0.003 已不再是自动有效的合同：应把新的能量漂移包络、 E_f 与 E_p 的分量曲线作为一份数值账本；再把速度矩/分布、谱或相关函数，或与解析解、收敛研究、实验量的比较作为独立的统计或物理账本。只有两本账分别闭合，才可以分别讨论守恒误差与统计噪声。

这张卡的源码证据由官方 thermal-plasma test 的 CTest 注册、1D/2D 输入与 `analysis.py`，以及 ParticleEnergy、FieldEnergy reduced diagnostic 和官方参数/算法说明共同组成。

精确路径用于定位职责，而不是替代运行。静态核对不会执行该运行；真实改动仍须由读者运行新 producer，并为新 observable 选择 reference 与容差。

1.10 Debye 长度、粒子数与统计时间尺度

Birdsall 1985 对 sheet model 的讨论给了一个比教科书定义更适合写进程序书的视角。

首先，在线性、近 Maxwellian 的三维等离子体中，热速率 v_t 、等离子体频率和 Debye 长度可写为

$$v_t = \sqrt{\frac{k_B T_s}{m_s}}, \quad \omega_{ps} = \sqrt{\frac{n_s q_s^2}{\epsilon_0 m_s}}, \quad \lambda_{Ds} = \frac{v_t}{\omega_{ps}} = \sqrt{\frac{\epsilon_0 k_B T_s}{n_s q_s^2}}.$$

其中 n_s 是物种数密度， T_s 是温度， k_B 是 Boltzmann 常数。对应的 Debye 球内粒子数近似为

$$N_D \sim (4\pi/3)n_s \lambda_{Ds}^3.$$

这些量的精确定义会随单位制、速度矩约定、磁化程度和 reduced geometry 改变；这里使用它们是为了建立尺度判断，而不是把二维或轴对称计算机械地当成三维 Debye 球。Debye 长度 λ_D 和 Debye 球内粒子数 N_D 因而不是孤立的公式，而是“这个 plasma 是否能被当作 collective medium”与“统计噪声会以什么尺度渗入观测量”的共同边界。

其次，在 reduced model 下，

$$\tau \sim \frac{2N_D}{\omega_p}$$

更适合被理解成：

- randomization time
- correlation time
- 统计独立采样间隔

而不是整个分布完全热化成 Maxwellian 的总弛豫时间。后者通常更慢，量级更接近 N_D^2 。对 PIC 用户来说，这比“记住 Debye 长度定义”更实用，因为它直接影响：

- uniform-plasma 噪声底怎么看；
- reduced diagnostics 应平均多久；
- 弱效应、弱不稳定和 Landau damping 的 measurement window 多大才可信。

1.10.1 尺度计算卡：能量回归通过不等于 Debye 屏蔽已分辨

上一节的官方 energy_conserving_thermal_plasma producer 正好说明，测试名和物理判据必须分开读。其 1D 输入设定 $T_e = 100 \text{ eV}$ 、

$$d_{e0} = c/\omega_{pe}, \quad \Delta t = 0.2/\omega_{pe}, \quad L = 10d_{e0}, \quad N_x = 8.$$

同一输入把电子热动量宽度写成 $u_{th} = \sqrt{T_e q_e / m_e} / c$ 。沿用本节的非相对论热速率约定，便有

$$\frac{\lambda_{De}}{d_{e0}} = \frac{v_{te}}{c} = \sqrt{\frac{T_e}{m_e c^2}} = \sqrt{\frac{100}{510998.95}} \approx 1.399 \times 10^{-2}.$$

而网格与一步的无量纲尺度分别为

$$\frac{\Delta x}{d_{e0}} = \frac{10}{8} = 1.25, \quad \frac{\lambda_{De}}{\Delta x} \approx 1.119 \times 10^{-2}, \quad \omega_{pe} \Delta t = 0.2, \quad \frac{v_{te} \Delta t}{\Delta x} \approx 2.24 \times 10^{-3}.$$

先读这四个数各自回答的问题。最后一个数小，只说明一个电子热速率在一步内移动的距离远小于一个 cell；它不能把 $\lambda_{De} / \Delta x \ll 1$ 变成空间分辨。这里的 1D/2D producer 因而可以为其固定 gather、shape、边界和输出 cadence 检查能量账本，却不由该 consumer 证明 Debye 屏蔽、Langmuir 色散、Landau damping 或热涨落谱已经按目标物理分辨。EP/EF 的 **0.003** 断言也没有读取 λ_D 、谱峰或温度矩。

这不是对该 regression 的否定。它的可支持结论本来就是“在这份输入和已记录时刻，指定的粒子动能加场能变化不超过阈值”；用一个本来不测空间分辨率的 consumer 去给它附会分辨率结论，才是错误。相反，宏粒子数也要单独解释：1D 的每 cell 4 个、2D 的每 cell 2×2 个宏粒子是采样设置，不是三维真实 Debye 球内物理粒子数 N_D 。

若研究目标真的需要 Debye 尺度，建立新的分辨率合同。首先从目标物理决定需要比较的 $\lambda_D / \Delta x$ 与时间窗，而不是机械复用本例的 8 cells；随后在固定物理长度、边界、温度、密度与可观察量定义下进行网格和宏粒子数的独立扫描。consumer 至少应包含与问题匹配的 density/field spectrum、温度或速度分布矩、解析色散/阻尼率、收敛趋势或实验 reference 中的一项；同时继续保留 $E_f + E_p$ 作为另一条数值账本。只有“尺度分辨”和“能量漂移”分别有证据，才可以讨论计算是否既稳定又回答了原来的热等离子体问题。

这个计算卡只重新计算官方输入已声明的无量纲量，并不运行 WarpX，也不宣布某个通用的 $\lambda_D / \Delta x$ 通过线。不同几何、模型、可观察量和误差预算需要各自的 reference 与收敛计划。

1.11 从连续模型到 PIC 离散变量

前面的方程还没有直接变成程序里的数组。PIC 的第一步不是把分布函数存成一个高维网格，而是用带权粒子样本代表它，再用网格上的有限差分或谱变量承载场。可以把这条映射写成下面的最小对应关系：

连续对象	PIC 离散载体	典型时间层/位置	在 WarpX 代码中应如何理解
$f_s(x, p, t)$	物种粒子的位置、动量、权重集合	$x_p^n, p_p^{n-1/2}, w_p$	ParticleContainer 中的粒子样本，不是一个逐点存储的分布函数
$\rho(x, t)$	shape-weighted charge density	ρ^n 或 $\rho^{n+1/2}$	由粒子沉积得到；rho_fp 与 rho_buf 还可能分别属于 fine 与 coarse-buffer 路径

连续对象	PIC 离散载体	典型时间层/位置	在 WarpX 代码中应如何理解
$J(x, t)$	trajectory-based current density	通常跨越 $n- > n+1$	Esirkepov/Villasenor 等 current deposition 需要 old/new 轨迹或等价的 crossing 信息
E, B	staggered/collocated grid fields	由 solver 规定	Efield_*, Bfield_* 的后缀表示时间层、网格位置或辅助副本，不能只按变量名猜物理时刻
连续性方程	离散 source synchronization	每个粒子推进步或 solver stage	SyncCurrentAndRho(), guard exchange、AMR average-down 等共同完成可供场求解器使用的 source

最容易被忽略的是 ρ 和 J 的时间语义不同。单时间层的 charge deposition 可以直接对粒子位置取样：

$$\rho_i^n = (1/\Delta V_i) \sum_p q_p w_p S_i(x_p^n),$$

而守恒的 current deposition 必须表达粒子从 x_p^n 到 x_p^{n+1} 的输运：

$$\Delta t (\nabla_h \cdot J)_i = \rho_i^n - \rho_i^{n+1}.$$

这里的第二式不是说所有 current deposition 都在程序中显式先算出左右两边，而是说明它们必须满足同一条离散连续性关系。也因此，

- DepositCharge() 负责单时间层的 ρ 采样及其时间层、几何和 AMR 桥接；
- DepositCurrent() 及其 Esirkepov/Villasenor 等路径负责轨迹输运产生的 J ；
- SyncCurrentAndRho() 负责把不同 level、边界和 source buffer 中的结果整理成 solver 可消费的源项。

这三层不能合并成“粒子把电荷写到网格”一句话。后续第 5 章会从 kernel 角度展开，第 6 章则会继续说明不同 field solver 如何消费这些 source。附录 A 给出 rho_fp、rho_buf、current_fp、current_buf 和 lev 等 WarpX 实现变量的速查定义。

1.12 这一章对后面源码章节的真正约束

到这里，后续读 WarpX 代码时至少要带着下面这些硬问题，而不是只盯函数名：

1. 粒子推进器是否在离散时间层上合理近似了 Liouville 流？
2. 沉积算法是否把连续性方程离散闭合到了 rho/J？

3. field solver 处理的是 Maxwell 还是 Poisson, 约束方程怎样传播?
4. shape factor 和 finite-size particles 是如何改写噪声、aliasing 和 self-force 的?
5. diagnostics 到底在测真实物理量, 还是只看离散噪声底的一个投影?

如果没有这几层边界, 后面源码里的:

- OneStep_nosub
- PushParticlesandDeposit
- SyncCurrentAndRho
- PushPSATD
- ElectrostaticSolver
- ImplicitSolver

都会被读成“工程控制流”, 而不是“连续模型的离散化实现”。

1.12.1 跨章交接卡: 从连续对象进入离散循环

第 1 章回答“哪些物理对象必须被表示”; 第 2 章回答“这些对象在一个离散时间步的什么时刻交换”。因此, 进入第 2 章前, 不应只带着一张函数名清单, 而应先写出下列状态账本:

连续问题	必须保留的离散状态	进入第 2 章后首先核对	不能直接推出的结论
分布函数 f_s 怎样被近似	宏粒子的 x_p 、 p_p 、 w_p 与 shape	2.2 中粒子样本如何 gather、deposit	宏粒子数较多不自动保证相空间的每个细节都被分辨
ρ 与 $\mathbf{J}$ 怎样成为场的 source	单时间层 ρ^n 与跨轨迹的 $\mathbf{J}^{n+1/2}$	2.2–2.3 中连续性方程和 leapfrog 时间层	一次 charge deposition 不能替代守恒的 current deposition 或 source synchronization
$\mathbf{E}, \mathbf{B}$ 怎样作用于粒子又被 source 改写	网格位置、时间层和场表示	2.3–2.5 中交错 FDTD 或谱空间推进	“有 Maxwell 方程”不说明某个 stencil、边界或 solver 组合具有可接受色散
哪些尺度必须由网格和步长承担	ω_p 、 λ_D 、特征速度、 Δt 、 Δx	2.3 的时间层和可分辨尺度讨论	CFL 可通过不表示 $\omega_p \Delta t$ 、 $\lambda_D / \Delta x$ 或统计误差已经满足物理目标

这张表给出一个严格的交接顺序: 先说明粒子和场分别代表什么, 再标出它们所在的时间层, 最后才追踪程序调用。若不先区分“连续对象”“离散状态”和“可比较 observable”, 就会把稳定性、守恒性与物理分辨率误当成同一类判断。第 2 章开头的接读卡将把这四类状态排成一次 $n \rightarrow n+1$ 的实际交换顺序。

1.13 证据范围与继续阅读

本章下列论述可直接回到两部已提供逐段讲解的基础来源：

- Birdsall 1985: sheet model 的 randomization / correlation / thermalization 时间尺度，以及 finite-grid / aliasing / fluctuation / heating 主线。
- Dawson 1983: numerical experiment 视角、superparticle / weighted particles 的 kinetic 边界，以及 finite-size particles、网格和 FFT-Poisson 的 electrostatic contract。

下面两项适合作为后续补充阅读，但本章不依赖其未取得全文的细节：

- Hockney–Eastwood: 加权粒子、heating estimates 和 optimum path 的经典表述。
- Yee 1966: staggered FDTD 与离散约束传播的原始出处。

继续阅读时，应优先回到原始论文、教材或官方文档，并为每一条主张标明它属于连续模型、离散算法还是特定代码实现；不要用其中任一层代替另外两层。第 2 章会把这里的 leapfrog、CFL、Debye 长度和数值色散接到同一条离散主循环上。

1.14 练习与源码定位

1. 变量桥接题：根据 1.11 的映射表，说明为什么 ρ_{fp}/ρ_{buf} 不能直接当作两个不同物理量，并指出它们分别在哪个 AMR/source-synchronization 场景出现。
2. 尺度判断题：给定 $\lambda_D/\Delta x = 0.5$ 和 $v_t\Delta t/\Delta x = 1.2$ ，列出至少两个可能的数值风险，并说明它们分别属于空间分辨率、粒子跨单元输运还是时间推进约束。
3. 源码定位题：从 `Source/Evolve/WarpXEvolve.cpp` 的 `OneStep_nosub()` 开始，而不是从全局搜索结果中猜入口。完成下列三步，并交付一个三行表格（函数、连续对象、调用后可认为“已准备好”的数据）：
 - 记录 `PushParticlesandDeposit()` 与 `SyncCurrentAndRho()` 的相对次序；前者对应粒子输运和源项沉积，后者应使场求解前的 J/ρ 经滤波、guard-cell/MR 同步和边界处理而可被消费。
 - 打开 `WarpX::SyncCurrentAndRho()` 的定义，列出其中至少两项 source 数据准备动作，并说明它们为什么不能由连续方程本身自动保证。
 - 任选一个场更新分支：PSATD 路线继续定位 `PushPSATD()`（定义在 `Source/FieldSolver/WarpXPushFieldsEM.cpp`）；FDTD 路线则定位 `EvolveB()/EvolveE()` 的调用。说明该分支把哪一种离散 Maxwell 闭合推进到下一时间层。

第二章 PIC 总循环：从 Vlasov-Maxwell 到离散时间推进

本章先不急进入某一个 WarpX 函数。生产级 PIC 代码的困难不在于“有粒子、有网格、有 Maxwell 方程”这几个名词，而在于这些对象必须在离散时间层、离散空间布局、并行 guard cells、边界条件和守恒约束之间保持一致。后续逐行读 WarpX 时，本章给出判断代码是否“物理上在做正确事情”的基准。

本章给出的函数名和调用关系是读代码的定位方法，而不是某一份源码树的版本标签。下文的 Source/... 与 Examples/... 均相对于 WarpX 源码根目录；使用不同 WarpX 版本时，应先从 Evolve()、OneStep()、PushParticlesandDeposit() 和 SyncCurrentAndRho() 等职责明确的入口重新建立调用关系，而不要把行号或本机工作区布局当作稳定 API。

Yee 1966 在本书中只承担一个窄的历史定位：有限差分 Maxwell 方程通过合适的场点布置处理导体边界。它不能替代本章的 stencil、时间层或色散推导；这些内容由连续方程、离散推导和现代实现三层分别说明。CartesianYeeAlgorithm.H、FiniteDifferenceSolver.cpp、EvolveB.cpp 与 EvolveE.cpp 的交叉定位可供读者复查，但现代代码不构成对历史论文逐式等价的证明。

2.0.1 从第 1 章进入本章：先写状态，再读调用顺序

承接第 1 章的交接卡，本章把“连续对象的离散表示”排成一次真正的时间交换。读者第一次阅读时，先把下面四个问题写在同一页上：

1. 当前粒子状态是 $x^n, p^{n-1/2}, w$ ，还是某个初始化、碰撞或隐式迭代中的试探状态？
2. 当前 ρ 是对哪个位置时间层的取样， $\mathbf{J}$ 是否已经表示了 $x^n \rightarrow x^{n+1}$ 的粒子输运？
3. 场 $\mathbf{E}, \mathbf{B}$ 位于何种网格和时间层，它们在 gather、沉积和 field solve 前后分别被谁消费？
4. $\omega_p \Delta t$ 、 $\lambda_D / \Delta x$ 、CFL 和统计粒子数分别约束哪一种误差，而不是把它们缩成一个“稳定”标签？

可按如下顺序进入正文：先用 2.1–2.2 把 f_s 、粒子、shape、 ρ 与 $\mathbf{J}$ 对齐；再用 2.3 写清整数步、半整数步和可分辨尺度；随后用 2.4–2.5 区分场更新表示；最后用 2.6 将这些状态接入真实 PIC step。这样，PushParticlesandDeposit()、SyncCurrentAndRho() 与场推进函数才是状态转换的实现，而不是可以脱离物理和时间层独立记忆的调用顺序。

2.1 连续模型：Vlasov-Maxwell 系统

对物种 s ，相空间分布函数 $f_s(\mathbf{x}, \mathbf{p}, t)$ 满足 Vlasov 方程：

$$\frac{\partial f_s}{\partial t} + \mathbf{v} \cdot \nabla_{\mathbf{x}} f_s + q_s (\mathbf{E} + \mathbf{v} \times \mathbf{B}) \cdot \nabla_{\mathbf{p}} f_s = 0.$$

相对论动量与速度满足

$$\mathbf{p} = \gamma m_s \mathbf{v}, \quad \gamma = \sqrt{1 + \frac{|\mathbf{p}|^2}{m_s^2 c^2}}, \quad \mathbf{v} = \frac{\mathbf{p}}{\gamma m_s}.$$

电磁场满足 Maxwell 方程：

$$\frac{\partial \mathbf{B}}{\partial t} = -\nabla \times \mathbf{E},$$

$$\frac{\partial \mathbf{E}}{\partial t} = c^2 \nabla \times \mathbf{B} - \frac{\mathbf{J}}{\epsilon_0},$$

以及约束方程

$$\nabla \cdot \mathbf{E} = \frac{\rho}{\epsilon_0}, \quad \nabla \cdot \mathbf{B} = 0.$$

源项来自分布函数的矩：

$$\rho(\mathbf{x}, t) = \sum_s q_s \int f_s(\mathbf{x}, \mathbf{p}, t) d\mathbf{p},$$

$$\mathbf{J}(\mathbf{x}, t) = \sum_s q_s \int \mathbf{v}(\mathbf{p}) f_s(\mathbf{x}, \mathbf{p}, t) d\mathbf{p}.$$

PIC 的核心任务就是把这套连续耦合系统离散成两个相互交换信息的对象：宏粒子和网格场。

2.2 宏粒子表示与形函数

宏粒子近似把 f_s 写成有限个带权粒子的和：

$$f_s(\mathbf{x}, \mathbf{p}, t) \approx \sum_{p \in s} w_p S_x(\mathbf{x} - \mathbf{x}_p(t)) S_p(\mathbf{p} - \mathbf{p}_p(t)).$$

这里 w_p 是宏粒子权重， S_x 是空间形函数。把粒子源项沉积到网格单元或网格点 i ，常见电荷密度形式是

$$\rho_i^n = \frac{1}{\Delta V_i} \sum_p q_p w_p S_i(\mathbf{x}_p^n).$$

场 gather 则使用同一类形函数从网格插值回粒子位置：

$$\mathbf{E}_p^n = \sum_i S_i(\mathbf{x}_p^n) \mathbf{E}_i^n, \quad \mathbf{B}_p^n = \sum_i S_i(\mathbf{x}_p^n) \mathbf{B}_i^n.$$

如果只看这两个公式，很容易以为 PIC 的网格-粒子耦合只是“双向插值”。这个理解不够。电流 $\mathbf{J}$ 的沉积必须表达粒子在一个时间步内的轨迹，否则离散电荷守恒会被破坏。

连续系统满足连续性方程：

$$\frac{\partial \rho}{\partial t} + \nabla \cdot \mathbf{J} = 0.$$

在网格上，电流沉积应尽量满足对应的离散形式：

$$\frac{\rho_i^{n+1} - \rho_i^n}{\Delta t} + (\nabla_h \cdot \mathbf{J}^{n+1/2})_i = 0.$$

这解释了为什么 WarpX 这样的代码会提供 Esirkepov、Villasenor-Buneman、Vay 等沉积分支。它们的差异不是表面上的“把电流放到哪里”，而是如何在离散网格上把粒子轨迹、电荷守恒、网格布局和并行边界结合起来。

2.3 leapfrog 时间层

显式电磁 PIC 的标准时间层是 leapfrog：

- 粒子位置在整数步： $\mathbf{x}^n, \mathbf{x}^{n+1}$ 。
- 粒子动量在半整数步： $\mathbf{p}^{n-1/2}, \mathbf{p}^{n+1/2}$ 。
- 电流自然在半整数步： $\mathbf{J}^{n+1/2}$ 。
- 电磁场在 Yee/FDTD 路径中按半步磁场和整步电场交错推进。

粒子位置推进可写成

$$\frac{\mathbf{x}^{n+1} - \mathbf{x}^n}{\Delta t} = \mathbf{v}^{n+1/2}.$$

动量推进写成抽象形式：

$$\frac{\mathbf{p}^{n+1/2} - \mathbf{p}^{n-1/2}}{\Delta t} = q (\mathbf{E}_p^n + \bar{\mathbf{v}} \times \mathbf{B}_p^n).$$

其中 $\bar{\mathbf{v}}$ 的具体定义取决于 pusher。Boris、Vay、Higuera-Cary 等 pusher 的差别留到粒子推进章节逐行讲解。本章只强调主循环必须给 pusher 提供正确时间层的 $\mathbf{x}$ 、 $\mathbf{p}$ 、 $\mathbf{E}$ 、 $\mathbf{B}$ 。

WarpX 的显式无 subcycling 路径在 `Source/Evolve/WarpXEvolve.cpp` 的 `WarpX::OneStep_nosub()` 开头直接把这个时间层写进注释：

```
Push particle from  $\mathbf{x}^{\{n\}}$  to  $\mathbf{x}^{\{n+1\}}$ 
                    from  $\mathbf{p}^{\{n-1/2\}}$  to  $\mathbf{p}^{\{n+1/2\}}$ 
Deposit current  $\mathbf{j}^{\{n+1/2\}}$ 
Deposit charge density  $\rho^{\{n\}}$ 
```

这四行是读 WarpX 主循环的锚点。任何 field gather、collision、ionization、deposition、sync、field solve 的位置都应围绕这些时间层理解。

2.3.1 ω_p 不是背景常数，而是时间离散必须尊重的最快等离子体尺度

对电子等离子体，最基本的本征时间尺度是 plasma frequency：

$$\omega_p = \sqrt{\frac{n_e e^2}{m_e \epsilon_0}}.$$

它决定了最简单的 Langmuir 振荡周期

$$T_p = \frac{2\pi}{\omega_p}.$$

对显式 leapfrog PIC，稳定和分辨不是同一件事。只要时间层排列正确、场更新满足 CFL，代码也许不会立刻炸掉；但如果

$$\omega_p \Delta t$$

已经接近或超过 1 的量级，那么单步内粒子和场已经跨过了 plasma oscillation 的核心相位结构，后面看到的高噪声、相位误差、非物理 heating，往往不是“分析脚本太苛刻”，而是主循环本身没有分辨这个最快内禀尺度。

这也是为什么 Birdsall 1985 后面会把

$$\omega_p \Delta t, \quad v_t \Delta t / \Delta x$$

都写成数值健康度的第一层控制量。对 WarpX 来说，这条边界不会自动由 `ComputeDt()` 替你保证。`ComputeDt()` 只根据 `solver/CFL`、`const_dt`、`max_dt`、`maxParticleVelocity()` 和 AMR refinement 给出一个可运行步长；它并不知道你要不要精确分辨 Langmuir 振荡、electrostatic shielding 或弱不稳定增长率。

所以本章这里先压实一个最重要的判断：

- `ComputeDt()` 保证的是一层离散稳定性和时间步组织约束；
- $\omega_p \Delta t$ 是否足够小，仍然是物理建模和分辨率设计问题。

2.3.2 λ_D 不只是一条长度定义，它直接约束 Δx

和 ω_p 对偶的空间尺度是 Debye length。对非相对论热电子，

$$\lambda_D = \sqrt{\frac{\epsilon_0 k_B T_e}{n_e e^2}} = \frac{v_{th,e}}{\omega_p}.$$

这条式子把热速度、plasma frequency 和 shielding length 绑在了一起。对 PIC 而言，能否把 plasma 当作 collective medium 与网格是否真的分辨了 shielding 不是分开的两个问题。

如果

$$\Delta x \gg \lambda_D,$$

那么 cell 内已经把最基本的 shielding 结构粗化掉了。接下来即使宏观波形看起来还能跑，field fluctuation、aliasing、self-force 和 nonphysical collisionality 也会被系统性放大。这就是为

什么第 1 章已经把 λ_D 、 N_D 和统计时间尺度单独拎出来；在第 2 章里，它进一步变成主循环的硬分辨率边界：

- Δt 决定是否分辨 ω_p ；
- Δx 决定是否分辨 λ_D ；
- 两者一起决定 leapfrog + grid PIC 到底是在近似同一个 plasma，还是已经换成了另一个更噪、更热、更强 alias 的离散模型。

2.4 FDTD 场更新的数学骨架

忽略 PML、divergence cleaning、宏观介质和边界时，Yee/FDTD 的主更新可以写成三段：

$$\mathbf{B}^{n+1/2} = \mathbf{B}^n - \frac{\Delta t}{2} \nabla_h \times \mathbf{E}^n,$$

$$\mathbf{E}^{n+1} = \mathbf{E}^n + c^2 \Delta t \nabla_h \times \mathbf{B}^{n+1/2} - \frac{\Delta t}{\epsilon_0} \mathbf{J}^{n+1/2},$$

$$\mathbf{B}^{n+1} = \mathbf{B}^{n+1/2} - \frac{\Delta t}{2} \nabla_h \times \mathbf{E}^{n+1}.$$

这说明一个电磁 PIC step 的顺序不能随意交换。电场更新需要本步沉积出来的 $\mathbf{J}^{n+1/2}$ ，所以粒子推进与电流沉积必须在 EvolveE(dt) 之前完成。WarpX 的 FDTD 路径正是这样组织的：

对应的核心源码节选来自 Source/Evolve/WarpXEvolve.cpp 的 WarpX::OneStep_nosub()，这里保留源项同步和 FDTD 场推进部分；PSATD 分支和 PML 后处理在第 3 章继续展开：

```
// Synchronize J and rho:
// filter (if used), exchange guard cells, interpolate across MR levels
// and apply boundary conditions
SyncCurrentAndRho();

// For extended PML: copy J from regular grid to PML, and damp J in PML
if (do_pml && pml_has_particles) { CopyJPML(); }
if (do_pml && do_pml_j_damping) { DampJPML(); }

ExecutePythonCallback("beforeEsolve");

EvolveF(0.5_rt * dt[0], /*rho_comp=*/0);
EvolveG(0.5_rt * dt[0]);
FillBoundaryF(guard_cells.ng_FieldSolverF);
FillBoundaryG(guard_cells.ng_FieldSolverG);

EvolveB(0.5_rt * dt[0], SubcyclingHalf::FirstHalf, a_cur_time); // We
↪ now have B^{n+1/2}
FillBoundaryB(guard_cells.ng_FieldSolver, WarpX::sync_nodal_points);

if (m_em_solver_medium == MediumForEM::Vacuum) {
    // vacuum medium
    EvolveE(dt[0], a_cur_time); // We now have E^{n+1}
} else if (m_em_solver_medium == MediumForEM::Macroscopic) {
```

```

    // macroscopic medium
    MacroscopicEvolveE(dt[0], a_cur_time); // We now have E^{n+1}
} else {
    WARPX_ABORT_WITH_MESSAGE("Medium for EM is unknown");
}
FillBoundaryE(guard_cells.ng_FieldSolver, WarpX::sync_nodal_points);

EvolveF(0.5_rt * dt[0], /*rho_comp=*/1);
EvolveG(0.5_rt * dt[0]);
EvolveB(0.5_rt * dt[0], SubcyclingHalf::SecondHalf, a_cur_time + 0.5_rt
↪ * dt[0]); // We now have B^{n+1}

```

数学动作	WarpX 源码位置
粒子从 $\mathbf{x}^n, \mathbf{p}^{n-1/2}$ 推到 $\mathbf{x}^{n+1}, \mathbf{p}^{n+1/2}$, 并沉积源项	Source/Evolve/WarpXEvolve.cpp: WarpX::OneStep_nosub() 对 PushParticlesandDeposit() 的调用
同步 $\rho, \mathbf{J}$: 滤波、guard cells、AMR、边界 F, G 半步、 $\mathbf{B}$ 半步	同一函数中的 SyncCurrentAndRho() 同一函数 FDTD 分支中的 EvolveF()、EvolveG()、EvolveB(...FirstHalf...)
$\mathbf{E}$ 整步	同一分支中的 EvolveE() 或 MacroscopicEvolveE()
F, G 半步、 $\mathbf{B}$ 半步	同一分支中的第二个 EvolveF()、EvolveG()、EvolveB(...SecondHalf...)
PML 与 guard cell 处理	同一分支中的 DampPML() 与 FillBoundary*()

这里的 F, G 是 divergence cleaning 相关辅助场，不是最小 Maxwell 更新必需项。WarpX 把它们插在场推进两侧，是为了在实际模拟中控制 $\nabla \cdot \mathbf{E}$ 或 $\nabla \cdot \mathbf{B}$ 误差及 PML 相关处理。

2.4.1 CFL 不是经验参数，而是离散 Maxwell 更新的因果上界

WarpX 的 ComputeDt() 不会把 FDTD 时间步写死成 $\min(\Delta x_i)/c$ ，而是把这件事委托给具体差分算法。Source/Evolve/WarpXComputeDt.cpp 的 WarpX::ComputeDt() 直接在主类层完成 solver 分派：

```

} else if (electromagnetic_solver_id ==
↪ ElectromagneticSolverAlgo::PSATD) {
    deltat = cfl * minDim(dx) / PhysConst::c;
} else {
    #if defined(WARPX_DIM_RZ) || defined(WARPX_DIM_RCYLINDER)
        if (electromagnetic_solver_id == ElectromagneticSolverAlgo::Yee) {
            deltat = cfl * CylindricalYeeAlgorithm::ComputeMaxDt(dx,
↪ n_rz_azimuthal_modes);
        }
    #elif defined(WARPX_DIM_RSPHERE)
        if (electromagnetic_solver_id == ElectromagneticSolverAlgo::Yee) {
            deltat = cfl * SphericalYeeAlgorithm::ComputeMaxDt(dx);
        }
    #else
        if (grid_type == GridType::Collocated) {
            deltat = cfl * CartesianNodalAlgorithm::ComputeMaxDt(dx);
        } else if (electromagnetic_solver_id ==
↪ ElectromagneticSolverAlgo::Yee

```

```

        || electromagnetic_solver_id ==
        ↪ ElectromagneticSolverAlgo::ECT) {
            deltat = cfl * CartesianYeeAlgorithm::ComputeMaxDt(dx);
    } else if (electromagnetic_solver_id ==
    ↪ ElectromagneticSolverAlgo::CKC) {
            deltat = cfl * CartesianCKCAlgorithm::ComputeMaxDt(dx);

```

对标准 Cartesian Yee，真正的 CFL 上界由 Source/FieldSolver/FiniteDifferenceSolver/FiniteDifferenceAlgorithms/CartesianYeeAlgorithm.H 中的 CartesianYeeAlgorithm::ComputeMaxDt() 给出：

```

static amrex::Real ComputeMaxDt ( amrex::Real const * const dx ) {
    using namespace amrex::literals;
    amrex::Real const delta_t = 1._rt / ( std::sqrt( AMREX_D_TERM(
        1._rt / (dx[0]*dx[0]),
        + 1._rt / (dx[1]*dx[1]),
        + 1._rt / (dx[2]*dx[2])
    ) ) * PhysConst::c );

    return delta_t;
}

```

也就是

$$\Delta t_{\max}^{\text{Yee}} = \frac{1}{c \sqrt{\Delta x^{-2} + \Delta y^{-2} + \Delta z^{-2}}}.$$

它表达的是离散光锥约束：单步内，Yee curl stencil 不能让信息传播得比离散网格所允许的因果速度更快。warpx.cfl 只是把这个严格上界再乘一个安全系数，不是“拍脑袋调参”。

2.4.2 数值色散从这里开始：连续光锥被离散 stencil 改写

一旦用有限差分近似 curl，连续真空色散关系

$$\omega^2 = c^2 |\mathbf{k}|^2$$

就不再原样保留。以 Yee 为例，差分导数对应的是

$$k_d \rightarrow \frac{2}{\Delta d} \sin \frac{k_d \Delta d}{2},$$

于是离散色散关系变成近似的

$$\sin^2 \frac{\omega \Delta t}{2} = c^2 \Delta t^2 \sum_d \frac{\sin^2(k_d \Delta d / 2)}{\Delta d^2}.$$

这意味着 phase velocity 和 group velocity 都会偏离连续值，且偏离大小依赖传播方向、波数和网格各向异性。数值色散不是后处理图里才会出现的现象，它在 UpwardDx() / DownwardDx() 这种最基本的差分定义处就已经写进去了。

Yee 的 x 向导数由同一文件中的 CartesianYeeAlgorithm::UpwardDx() 与 DownwardDx() 实现：

```

static amrex::Real UpwardDx (
    amrex::Array4<amrex::Real const> const& F,
    amrex::Real const * const coefs_x, int const /*n_coefs_x*/,
    int const i, int const j, int const k, int const ncomp=0 ) {
    ...
    amrex::Real const inv_dx = coefs_x[0];
    return inv_dx*( F(i+1,j,k,ncomp) - F(i,j,k,ncomp) );
}

template< typename T_Field>
static amrex::Real DownwardDx (
    T_Field const& F,
    amrex::Real const * const coefs_x, int const /*n_coefs_x*/,
    int const i, int const j, int const k, int const ncomp=0 ) {
    ...
    amrex::Real const inv_dx = coefs_x[0];
    return inv_dx*( F(i,j,k,ncomp) - F(i-1,j,k,ncomp) );
}

```

也就是 staggered grid 上的前/后向一阶差分:

$$D_x^+ F_i = \frac{F_{i+1} - F_i}{\Delta x}, \quad D_x^- F_i = \frac{F_i - F_{i-1}}{\Delta x}.$$

这里的 Upward/Downward 不只是“正向/反向”, 而是在 nodal 与 cell-centered 位置之间搬运离散导数。正是这种 staggered 几何, 让 Yee 在保持二阶精度的同时把 E/B 交错布置起来。

2.4.3 Yee / Nodal / CKC 的差别本质上是离散色散关系不同

对 collocated/nodal solver, WarpX 的 CartesianNodalAlgorithm 不再用 staggered 前后差分, 而是直接用中心差分。可从 Source/FieldSolver/FiniteDifferenceSolver/FiniteDifferenceAlgorithms/CartesianNodalAlgorithm.H 的 UpwardDx() 与 DownwardDx() 回查:

```

static amrex::Real UpwardDx (
    amrex::Array4<amrex::Real const> const& F,
    amrex::Real const * const coefs_x, int const /*n_coefs_x*/,
    int const i, int const j, int const k, int const ncomp=0 ) {
    ...
    Real const inv_dx = coefs_x[0];
    return 0.5_rt*inv_dx*( F(i+1,j,k,ncomp) - F(i-1,j,k,ncomp) );
}

static amrex::Real DownwardDx (
    amrex::Array4<amrex::Real const> const& F,
    amrex::Real const * const coefs_x, int const n_coefs_x,
    int const i, int const j, int const k, int const ncomp=0 ) {
    ...
    return UpwardDx( F, coefs_x, n_coefs_x, i, j, k, ncomp);
}

```

所以 nodal grid 上 UpwardDx 和 DownwardDx 等价, 说明这里已经没有 Yee 那种 staggered 位置语义, 而是一个 collocated 中心差分系统。它的 CFL 上界虽然在当前实现里和 Yee 一样都是

$$\Delta t_{\max} \sim \frac{1}{c \sqrt{\sum_d \Delta d^{-2}}},$$

但数值色散与奇偶模耦合特征已经不同。

CKC 则更进一步，不再满足“一个方向只看一对最近邻”的局部导数定义。可从 `Source/FieldSolver/FiniteDifferenceSolver/FiniteDifferenceAlgorithms/CartesianCKCAlgorithm.H` 的 `ComputeMaxDt()` 与 `UpwardDx()` 回查：

```
static amrex::Real ComputeMaxDt ( amrex::Real const * const dx ) {
  #if (defined WARPX_DIM_1D_Z)
    amrex::Real const delta_t = dx[0]/PhysConst::c;
  #elif (defined WARPX_DIM_XZ)
    amrex::Real const delta_t = std::min( dx[0], dx[1] )/PhysConst::c;
  #else
    amrex::Real const delta_t = std::min( dx[0], std::min( dx[1],
      ↪ dx[2] ) ) / PhysConst::c;
  #endif
  return delta_t;
}

...
return alphax * (F(i+1,j ,k ,ncomp) - F(i, j, k ,ncomp))
  + betaxy * (F(i+1,j+1,k ,ncomp) - F(i ,j+1,k ,ncomp))
  + F(i+1,j-1,k ,ncomp) - F(i ,j-1,k ,ncomp))
  + betaxz * (F(i+1,j ,k+1,ncomp) - F(i ,j ,k+1,ncomp))
  + F(i+1,j ,k-1,ncomp) - F(i ,j ,k-1,ncomp))
```

这说明 CKC 的真实目标不是“换一套写法”，而是通过更宽的横向耦合 stencil 改写离散色散关系，从而改善高方向性传播下的 phase error。代价则是：

- stencil 更宽；
- 算法/geometry 适用范围更窄；
- guard-cell 和边界处理的组合空间更复杂。

2.5 PSATD 与 FDTD 的主循环差异

FDTD 在实空间用局部 stencil 近似 curl。PSATD 则在谱空间解析积分线性 Maxwell 方程的一部分。物理方程相同，但离散算法不同：

- FDTD 的优势是局部、显式、边界和 AMR 工程路径相对直接。
- PSATD 能显著降低数值色散，适合相对论束流、激光等问题，但对 FFT、并行 domain decomposition、边界、current correction 和时间平均场有更复杂要求。

这条分界线和前面几节正好连起来：

- leapfrog 规定了粒子、场和源项的时间层关系；
- ω_p 与 λ_D 规定了 plasma 自身是否被给定的 $\Delta t/\Delta x$ 分辨；
- CFL 规定了 Maxwell 更新是否还能保持离散因果；

- Yee/Nodal/CKC/PSATD 则进一步决定同一组 $\Delta t, \Delta x$ 会把波动相速度、群速度和 aliasing 改写成什么样。

所以“主循环能跑”不等于“主循环近似的是对的物理系统”。真正的 PIC loop 要同时满足：

time-layer consistency + charge/source consistency
+ physical scale resolution
+ solver-dependent stability/dispersion control.

WarpX 在 `OneStep_nosub()` 内部把两者清楚分开：

- PSATD 分支在 `Source/Evolve/WarpXEvolve.cpp` 的 `WarpX::OneStep_nosub()` 内，核心是 `PushPSATD(a_cur_time)`。
- FDTD 分支在同一函数内，核心是 `EvolveB/EvolveE/EvolveB`。

这意味着本书后续讲 field solver 时不能把“Maxwell solver”写成单一算法。`algo.maxwell_solver` 的选择会改变主循环内的场推进、同步、边界和可用功能。

2.6 一个完整 PIC step 的离散组织

把物理动作映射到程序实现，一个时间步至少包含这些层次：

1. 用户 callback、信号、诊断、负载均衡和步长更新。
2. 场 gather 前的 guard cell 与 auxiliary field 准备。
3. 电离、QED、粒子注入等改变粒子集合的多物理模块。
4. 粒子推进、碰撞、沉积电流和电荷。
5. 源项同步：滤波、guard cells、AMR fine/coarse 交换、边界条件。
6. 场推进：FDTD、PSATD、implicit、electrostatic 或 hybrid。
7. PML、moving window、粒子边界、重分布、排序。
8. 诊断写出和终止条件检查。

WarpX 在 `Source/Evolve/WarpXEvolve.cpp` 的 `WarpX::Evolve()` 中正是围绕这些层次组织外层循环。真正的主循环不是教科书五行伪代码，而是把守恒离散化、时间层一致性和大规模并行工程组合起来的控制流。

读者的时间层判断卡

遇到一个包含 AMR、JRhom 或 implicit 的输入时，先不要按函数出现次数数“执行了多少步”。先写出外层物理时间区间 $t^n \rightarrow t^{n+1}$ ，再给每次内部重复贴上唯一标签：细层的真实推进、source 的时间积分，或非线性求解的试探。最后才检查该路径在下一次 gather 前交出了哪一个 ρ 、 $\mathbf{J}$ 、 $\mathbf{E}$ 、 $\mathbf{B}$ 时间层。这个顺序能防止把 subcycling、JRhom 与 implicit 都误读成同一种“多调用几次”的算法。

2.6.1 AMR subcycling: 两个时间步不是同一个时间步的重复调用

无 subcycling 时, 第 0 层和更细层使用同一个外层时间步, `OneStep_nosub()` 可以把粒子推进、source synchronization 和场推进看成一条统一的 $n \rightarrow n+1$ 链。打开 subcycling 后, 这个图像不再成立。`OneStep_sub1()` 的这一路径明确限定: 只支持两级 mesh refinement, 且每个方向的 refinement ratio 必须为 2。

令粗层时间步为 Δt_c , 细层时间步为

$$\Delta t_f = \frac{\Delta t_c}{2}.$$

一个粗层周期内的基本推进结构是:

阶段	细层	粗层/母网格	源项职责
第一个半周期	推进一次粒子和场, 步长 Δt_f	暂不完成整步	将 <code>current_fp</code> 、 <code>rho_fp</code> 限制到 coarse patch
中间同步	细层已到 $t + \Delta t_f$	粗层推进到相应中间时间	合并细层、粗层和 buffer 中的源项
第二个半周期	再推进一次粒子和场, 步长 Δt_f	继续完成粗层剩余半步	第二段细层源项限制/合并后参与粗层场更新
粗层周期末	到 $t + \Delta t_c$	到 $t + \Delta t_c$	粗细层的场、源项和 guard cells 重新同步

因此, subcycling 不是简单地把 `OneStep_nosub()` 调两次。粗层粒子只推进一次, 而细层粒子推进两次; 粗层场的更新还要消费两个细层时间片上累积并平均/合成后的电流。源码中的调用顺序可以压缩成:

```
fine push/deposit at  $t \rightarrow$  restrict source  $\rightarrow$  fine field half/full/half,
coarse push/deposit at  $t \rightarrow$  combine coarse+fine source  $\rightarrow$  coarse field advance,
fine push/deposit at  $t + \Delta t_f \rightarrow$  restrict source  $\rightarrow$  fine field half/full/half,
combine second fine source  $\rightarrow$  coarse field completion.
```

这里的 `restrict` 和 `add` 不能与普通 guard-cell exchange 混为一谈: 前者改变的是 coarse/fine source 的层级表示, 后者只是同一层相邻 patch 间的数据可见性。对电荷来说, `rho_buf` 还可能来自 transition-zone 粒子在 coarse 几何上的直接沉积; 因此 subcycling 中的 source 合成既不是“把 fine rho 全部平均下来”这么简单, 也不是由场求解器自动补齐。

`OneStep_sub1()` 开头的断言拒绝 electrostatic solver 与 subcycling 的组合。原因不是 electrostatic 不能使用 AMR, 而是这条 subcycling 例程的时间组织只为显式 electromagnetic field advance 编写, 不能把 Poisson/electrostatic 路径的源项和场解时序悄悄套进来。

所以读 AMR PIC loop 时要同时检查四个不变量:

1. 细层是否确实使用 $\Delta t_c/2$, 且在一个粗层周期内推进两次;
2. 粗层粒子是否只推进一次, 避免重复计数粒子输运;
3. 两个细层时间片的 current/rho 是否分别完成 restrict、边界合并和 coarse source add;

4. 粗细层字段与 auxiliary/guard data 是否在下一 gather 前重新可见。

这四项共同构成 AMR subcycling 的时间一致性条件。缺少其中任何一项，都可能得到“每个 patch 都成功更新”但跨层电荷守恒、场相位或粒子 gather 已经不一致的结果。

2.6.2 JRhom 与 implicit: 同一个外层 step 内部也可能有不同的时间合同

标准 OneStep_nosub()、PSATD-JRhom 和 implicit solver 都可能被外层 WarpX::OneStep() 视为一次迭代，但它们内部对“源项在什么时候被求值”的定义不同。从 WarpX::OneStep() 的分派和各路径的函数职责看，应按下面三条路径分别阅读：

1. 标准显式 **electromagnetic PIC**。入口是 OneStep_nosub()；一次粒子推进产生 J 与 ρ ，随后统一进入 SyncCurrentAndRho()。场端是 FDTD 的 B-E-B 或一次 PSATD 推进；普通显式 collision placement 可以按各自时间层与它组合。
2. **PSATD-JRhom**。入口是 OneStep_JRhom()；它先推进粒子但跳过普通沉积，再按 ρ/J 的时间依赖在 Δt 内做多次相对时间沉积。每个 deposit interval 都执行一次谱空间场推进，并可选跨 $2\Delta t$ 时间平均；该路径只支持 PSATD，且不支持 current_correction 与 split momentum collision push。
3. **implicit electromagnetic PIC**。入口是 ImplicitSolver::OneStep()；以 $E^{n+\theta}$ 或中间场为猜测，在非线性/线性 RHS 评估中反复推进粒子并构造 $J^{n+1/2}$ 。nonlinear solver 先求自洽中间电场，再完成粒子和场的后半步；一次 RHS 评估不是一次物理时间步，mass-matrix/JFNK 也会改变 J 的构造路径。

JRhom 的关键不是“PSATD 多调用几次”，而是把时间依赖的源项显式建模成一组谱空间可消费的历史量。OneStep_JRhom() 的顺序可以压缩为：

1. 以 skip_deposition=true 把粒子从 $x^n, p^{n-1/2}$ 推到 $x^{n+1}, p^{n+1/2}$ ；
2. 把 E/B 以及可选的 divergence-cleaning 场变换到谱空间；
3. 按 time_dependency_rho 和 time_dependency_J 在相对时间上沉积 rho_new 与 J ，每次执行 filter、guard/AMR 同步和 Fourier transform；
4. 对每个子区间执行 PSATDPushSpectralFields()，最后把场变回实空间并处理 PML、边界和 guard cells。

其中 m_JRhom_subintervals 把一个外层步拆成 $\text{sub_dt} = \Delta t / n_{\text{deposit}}$ 的多个沉积区间；打开 time averaging 时，内部循环还会覆盖两个完整时间步的源/场积分。因而 JRhom 中的 ρ 和 J 不是标准显式路径里“本步各沉积一次”的同义词，不能直接用 OneStep_nosub() 的单点时间图解释它。

implicit 路径的差异更加根本。以 SemiImplicitEM::OneStep() 为例，程序先保存粒子和旧场，将磁场推进到半步，然后由 nonlinear solver 反复调用 ComputeRHS()；ImplicitSolver::PreRHSOp() 在每次 RHS 构造里：

- 用当前猜测的中间电场推进粒子；
- 形成半步电流；

- 在需要时把 `current_fp_non_suborbit`、mass matrices 或 JFNK 线性阶段的贡献合并进 J；
- 最后调用 `SyncCurrentAndRho()`，再把同步后的源项交给隐式残差/线性系统。

所以 implicit 中必须区分三个层次：

1. 物理时间步 $t^n \rightarrow t^{n+1}$ ；
2. 非线性迭代中的中间场猜测 $E^{n+\theta}$ ；
3. 每次 RHS 或 Jacobian 评估中的粒子/source 重算。

如果把第 3 层误写成“程序又推进了一个物理时间步”，就会错误理解粒子数、能量账本和 `SyncCurrentAndRho()` 的调用次数。相反，如果把 JRhom 的多个 deposit interval 当成 nonlinear iteration，也会把时间积分和求解器迭代混为一谈。

本章余下章节的阅读规则因此固定为：先识别外层物理时间步，再识别内部的 source subinterval 或 nonlinear iteration，最后才判断某次 `PushParticlesandDeposit()` 是物理推进、试探性 RHS 构造，还是历史源项重建。

读者可以用下表快速定位一个输入卡采用的时间组织：

优先判断	进入路径	这一外层步内部发生什么
m_implicit_solver 已创建	<code>ImplicitSolver::OneStep()</code>	多次 nonlinear iteration/RHS source rebuild，最后才提交一个物理时间步
psatd.JRhom 开启	<code>OneStep_JRhom()</code>	多个相对时间的 rho/J 沉积区间和对应的 PSATD 谱推进
AMR subcycling 开启	<code>OneStep_sub1()</code>	细层做两次 $dt/2$ 推进并限制/合并源项，粗层做一次 dt 推进
以上均否	<code>OneStep_nosub()</code>	push/deposit、 <code>SyncCurrentAndRho()</code> ，再进行 FDTD 或 PSATD 场推进

图中三种“内部多次执行”含义不同：implicit 的重复是求解器试探，JRhom 的重复是同一物理时间步内的源项时间积分，subcycling 的重复则是真实的细层物理推进。只有最后完成的外层调用才代表一次可提交的物理时间步。

2.7 本章后的源码阅读入口

读者现在可以从三个源码入口继续：

目标	入口
看外层时间步如何组织	<code>Source/Evolve/WarpXEvolve.cpp: WarpX::Evolve()</code>

目标	入口
看显式电磁无 subcycling 的标准 step	Source/Evolve/WarpXEvolve.cpp: WarpX::OneStep_nosub()
看主循环如何进入粒子容器	Source/Evolve/WarpXEvolve.cpp: WarpX::PushParticlesandDepos it()
看两级 AMR subcycling 的细/粗层时间组织	Source/Evolve/WarpXEvolve.cpp: WarpX::OneStep_sub1()
看 JRhom 的多次相对时间沉积与谱推进	Source/Evolve/WarpXEvolve.cpp: WarpX::OneStep_JRhom()
看 implicit RHS 中的粒子推进与 source syn- chronization	Source/FieldSolver/ImplicitSol vers/ImplicitSolver.cpp: Implic itSolver::PreRHSOp()

2.8 参数示例与最小运行案例

如果把本章压回一个最小、可运行、可验证的输入骨架，可从下面的官方案例开始：

- Examples/Tests/langmuir/inputs_test_1d_langmuir_multi

它把本章真正讨论的五类量都放在同一个最小问题上：

- geometry.dims = 1
- algo.current_deposition = esirkepov
- algo.field_gathering = energy-conserving
- warpx.cfl = 0.8
- max_step = 80
- 周期场边界

也就是说，本章的抽象讨论并不是悬空的。这里的：

- leapfrog 时间层
- ω_p
- λ_D
- FDTD curl 更新
- rho/J 连续性关系

都能在这条最小 Langmuir 主线上落到真实输入。

Examples/Tests/langmuir/CMakeLists.txt 将这张输入卡注册为 test_1d_langmuir_multi：它使用两个 MPI rank，并把末态 plotfile diags/diag1000080 交给 analysis_1d.py。这给出了一条读者可回查的闭环，而不是一串脱离上下文的历史数字：

1. 输入卡设置 `max_step = 80`、周期边界、`esirkepov` 沉积和第 40/80 步的 `full diagnostics`;
2. `analysis_1d.py` 从传入的末态 `plotfile` 读取 `Ez`，按脚本中的解析 `Langmuir` 解重建同一时刻的场，并要求 `error_rel < 0.05`;
3. 脚本随后调用 `analysis_utils.py` 的 `check_charge_conservation(data)`。对这张 `esirkepov`、非 PSATD、非 RZ 的输入，`helper` 实际检查

$$\frac{\max |\operatorname{div} E - \rho/\epsilon_0|}{\max |\rho/\epsilon_0|} < 10^{-11}.$$

这里的 `divE` 是诊断输出中的离散散度字段；这项 `gate` 检查的是该离散表示下的 Gauss-law/source 一致性，不是“任意几何、边界、粒子形函数和求解器组合都守恒”的证明。单次运行得到的具体误差会随构建、并行布局和案例修改而变化，因此读者应以分析脚本的定义和阈值为准，而不是把某次残差数值当成通用常数。

因此，完成这个案例的最低交付应包括：输入参数、所消费的末态诊断、解析场误差和离散 Gauss-law 误差分别说明什么，以及它们不能外推到哪些算法组合。这样才真正把连续模型、离散方程和可运行的验证量连起来。

2.8.1 第一次运行的读者路线：构建、CTest 与手动分析各自回答什么

上一节给出了一个可回查的案例，但“有输入文件”不等于“已按正确维度编译、以预期 rank 数运行并消费输出”。首次操作时，把下面三层分开，能避免绝大多数把构建错误、调度错误和物理误读混为一谈的问题。

第一层是构建能力。官方 CMake 文档说明维度是编译期选项，而当前源码允许 1、2、3、RZ、RCYLINDER 和 RSPHERE。本节的输入是 1D，因此建立一个明确的 1D build directory：

```
export WARPX_ROOT=/path/to/warpx
export WARPX_BUILD="$WARPX_ROOT/build-1d"

cmake -S "$WARPX_ROOT" -B "$WARPX_BUILD" -DWarpX_DIMS=1
cmake --build "$WARPX_BUILD" -j 4
```

不要把这个命令理解为“任意输入都能运行”。WarpX 的可执行文件名会编码编译选项；当前构建逻辑同时为 1D application 建立 `warpx.1d` 链接。因此，在继续前应确认实际二进制与输入的 `geometry.dims` 相容，而不是拿一个 3D、RZ 或旧 build directory 的可执行文件碰运气。

第二层是 CTest 参考执行。在默认 `WarpX_APP` 与 `BUILD_TESTING` 路径中，`Examples/` 会登记测试；`test_1d_langmuir_multi` 的 CMake 条目固定了 1D、2-rank、输入、分析脚本和 checksum。注意：这个名字是 **test set**，不是 CTest 的单个条目。

WarpX 的 `add_warpx_test()` 实际创建 `.run`、`.analysis` 和 `.checksum` 三项。因此先用 `-N` 只列出将匹配的项：

```
ctest --test-dir "$WARPX_BUILD" \
-N -R '^test_1d_langmuir_multi\.*'
```

正确配置时，这一步应列出：

```
test_1d_langmuir_multi.run
test_1d_langmuir_multi.analysis
test_1d_langmuir_multi.checksum
```

若总数为零，应先检查 build 是否启用了测试、是否包含 1D/MPI 能力，以及正则是否保留了字面量点号和后缀；零项不是一次通过的 **Langmuir** 验证。确认列表后，再执行同一组：

```
ctest --test-dir "$WARPX_BUILD" \
  -R '^test_1d_langmuir_multi\..*' \
  --output-on-failure
```

三项的职责和顺序也应分开读：`.run` 在 test working directory 以内按登记的两个 MPI rank 生成 producer 输出；`.analysis` 依赖 `.run`，调用 `analysis_1d.py diags/diag1000080` 检查解析场和离散 Gauss-law；`.checksum` 又依赖 `.analysis`，将指定输出交给默认回归 consumer。于是 `.run` 单独成功只能说明程序按该启动配置完成，不能代替解析场比较；反过来，三项共同通过也只支持这一条被登记的输入/consumer 合同，不等于已经对任意参数覆盖、任意硬件后端或完整 Langmuir 理论完成验证。

第三层是手动运行与复核。只有需要改变输入、保留自己的输出或观察中间 diagnostics 时，才绕开 CTest。应在新的运行目录中复制输入和分析器，而不是把 `diags/` 写进源码案例目录：

```
export RUN_DIR=$HOME/warpx-work/langmuir-1d
mkdir -p "$RUN_DIR"
cp "$WARPX_ROOT/Examples/Tests/langmuir/inputs_test_1d_langmuir_multi"
  ↪ "$RUN_DIR/inputs"
cp "$WARPX_ROOT/Examples/Tests/langmuir/analysis_1d.py" "$RUN_DIR/"
cp "$WARPX_ROOT/Examples/Tests/langmuir/analysis_utils.py" "$RUN_DIR/"

cd "$RUN_DIR"
mpirun -np 2 "$WARPX_BUILD/bin/warpx.1d" inputs
python analysis_1d.py diags/diag1000080
```

最后一行需要可用的 `yt`、`numpy`、`scipy` 和 `matplotlib` 分析环境；这与 C++ 可执行文件能否启动是两项不同前提。直接运行会生成 `warpx_used_inputs`，其中包含显式和隐式使用的参数；诊断默认写到 `diags/`。两者应与终端输出一起保存，才能说明此次运行实际使用了什么输入。若改动过 `max_step`、`diagnostics` 名称或输出周期，`diags/diag1000080` 未必仍存在，分析脚本和 consumer 路径也必须随之重新核对。

这三层形成一条首运行合同：先让编译维度匹配输入，再用 CTest 确认官方注册的 **producer/consumer** 路径，最后才在独立目录修改输入并重新定义 **analysis**。任何一层成功都不能替代另外两层；尤其是“程序退出为零”既自动产生解析误差 gate，也不自动证明手动修改后的输入仍满足原测试的结论范围。

2.8.2 受控修改路线：一个命令行覆盖会改变哪一份证据

第一次改输入时，最常见的错误不是参数语法，而是继续把原案例的 `analysis` 当成新运行的结论。WarpX 允许在命令行覆盖输入项，并把显式和隐式生效的参数写进 `warpx_used_inputs`；这使覆盖操作可追溯，却不让原来的 consumer 自动适配。

以本节 Langmuir 输入为例，`max_step = 80` 与 `diag1.intervals = 40` 共同产生第 40 与第 80 步的 Full diagnostics；CTest 登记的 `analysis_1d.py diags/diag1000080` 明确消费后者。下面的短运行只适合检查“命令行覆盖是否真正进入本次 producer”：


```
export PROBE_DIR=$HOME/warpx-work/langmuir-max-step-10
mkdir -p "$PROBE_DIR"
cp "$WARPX_ROOT/Examples/Tests/langmuir/inputs_test_1d_langmuir_multi"
  ↪ "$PROBE_DIR/inputs"

cd "$PROBE_DIR"
mpirun -np 2 "$WARPX_BUILD/bin/warpx.1d" inputs max_step=10 \
  > stdout.log 2>&1
rg '^max_step' warpx_used_inputs
```

这里 `warpx_used_inputs` 中的 `max_step = 10` 才是“覆盖已被消费”的证据；`stdout.log` 只能帮助定位启动、MPI 或参数错误。因为步数 10 没有达到这个输入的 40-step 输出间隔，原来注册的 `diags/diag1000080` 不会由此产生。故而下面这条命令在该 probe 上没有意义：

```
python analysis_1d.py diags/diag1000080
```

它失败或找不到文件不能说明 Langmuir 物理失败，只说明 consumer 仍指向原 producer 的末态。反过来，若手工预先留下同名旧目录，脚本甚至可能读取旧数据，造成更危险的假阳性。因此每个参数试验应有独立目录，且开始前确认 `diags/` 的时间步与本次 `warpx_used_inputs` 相符。

要重新获得一个有物理含义的比较，读者有两条不同的路线：

1. 保持官方合同。不改 `max_step`、`diag1.intervals`、场字段或解析模型参数，运行到 80 步，再把新生成的 `diags/diag1000080` 交给原 `analysis_1d.py`。这能重新检查指定 1D、2-rank 输入的解析场与离散 Gauss-law gate。
2. 建立新合同。若改动步数、diagnostic 周期、网格、沉积、边界或理论参数，先写明新的 producer 文件、consumer 读取字段、reference、容差与不可外推范围。`analysis_1d.py` 通过命令行参数取得文件名，并以文件内时间重建理论场；但这不自动证明原来的 0.05 阈值和 `check_charge_conservation` 的适用条件在修改后仍合理。

因此，一个最小的修改记录至少回答四个问题：哪一项输入被覆盖、`warpx_used_inputs` 如何证实它、这次实际写出了什么 `diagnostics`、哪个 consumer 因何仍然或不再适用。这比保存一串终端输出更接近可重复的数值实验，也把“配置变了”与“物理结论变了”分开。

2.9 基础文献与证据范围

本章直接依托的基础来源是：

- Birdsall 1985
 - leapfrog 最小教学骨架
 - $\omega_p \Delta t$
 - $v_t \Delta t / \Delta x$
 - finite-grid / aliasing / heating 主线
- * Dawson 1983

- electrostatic / full EM 的数值模型边界
- full EM 时间步与 light mode / CFL 的关系
- Darwin 作为 radiation-free low-frequency route

下列来源只作为继续阅读线索，不在本章中承担未核实的公式或数值结论：

- Yee 1966
- Hockney-Eastwood

继续扩展基础文献时，优先选择能够给出完整推导、适用假设和数值实验条件的原始教材或论文。书目线索本身只能帮助定位来源，不能代替公式、算法或案例结论的核查。

2.10 进一步阅读与练习

进一步阅读：

1. 第 3 章：WarpX 演化：把本章的 PIC loop 抽象结构接到 `main.cpp` $\rightarrow$ `WarpX::Evolve()` 的真实调用链。
2. Birdsall 1985：继续核对 $\omega_p \Delta t$ 、 $\lambda_D / \Delta x$ 、finite-grid aliasing 和 numerical heating 的条件与量纲。
3. Dawson 1983：继续比较 full EM、Darwin、quiet start 和 statistical measurements 如何改变“PIC 总循环”的解释方式。

练习题：

1. 解释为什么 `ComputeDt()` 给出的可运行时间步，不自动保证 $\omega_p \Delta t \ll 1$ 。
2. 用本章的 λ_D 讨论说明：为什么 $\Delta x \gg \lambda_D$ 时，即使主循环稳定，也可能已经不是同一个物理 plasma。
3. 结合本章的 Langmuir 案例，指出 `analysis_1d.py` 的两条核心断言分别对应本章哪两类理论边界。
4. 案例闭环题：依次读取 `inputs_test_1d_langmuir_multi`、`langmuir/CMakeLists.txt` 和 `analysis_1d.py`，交付一张四行表（输入设置、诊断 surface、分析断言、不可外推范围）。表中必须写清 `test_1d_langmuir_multi` 的两个 rank、`analysis_1d.py` `diags/diag1000080` 的绑定，以及为什么不能只从 `max_step = 80` 猜测分析器实际消费的输出路径和检查量。

2.11 本章结论

PIC 总循环的关键不在于依次调用“推粒子、沉积、推场”三个动作，而在于每个动作所使用的时间层和空间表示相容。读者应按以下顺序判断一条 PIC 路线：

1. 先确定连续问题与可分辨尺度。等离子体频率、Debye 长度、光波 CFL 和预期物理时间窗决定 `dt`、网格和粒子采样是否有机会描述目标问题。

2. 再确定粒子与网格的交换方式。gather、形函数和 ρ/J deposition 必须共同满足连续性和离散布局要求；仅有稳定的场更新不能补偿不相容的源项。
3. 区分外层时间步和内部重复。implicit nonlinear iteration、JRhom 的 source 时间积分与 AMR subcycling 都可能在一个外层步内多次执行，但它们分别代表试探、积分和真实细层推进，不能混用同一类验证量解释。
4. 最后用与问题匹配的 **observable** 验证。Langmuir 的解析场与 $\text{div}E - \rho/\epsilon_0$ 能检验指定输入下的波动和 Gauss-law 链；程序退出或 checksum 本身不能证明所有几何、边界和算法组合正确。

这条顺序把连续 Vlasov–Maxwell 模型连接到真实的离散时间推进，也为第 3 章的生命周期调用图、第 4、5 章的粒子/沉积链和第 6 章的场求解器选择提供共同的时间层坐标。